Vendor and Supplier Management System

Project Documentation

Voltbay: an electronics gadget store

1. Introduction

The Vendor and Supplier Management System (branded as Voltbay) is a full-stack web application for an electronics gadget store. The frontend is developed using React.js and the backend using Spring Boot. It allows an administrator to manage vendors, suppliers and products, and allows registered vendors to browse products and send purchase requests, which the administrator handles through quotations and purchase orders.

The application provides the following core operations:

- Admin and vendor login with role selection
- Register, view, edit, and delete vendors, with ACTIVE / INACTIVE status
- Add, view, edit, and delete suppliers
- Add, view, edit, and delete products with price and stock quantity
- Vendors place purchase requests for products
- Admin creates a quotation by selecting a supplier for a request
- Admin confirms or rejects purchase requests, with stock reduced on confirmation
- Vendors track the status of their requests and view confirmed quotations

2. Objectives

The main objectives of this project are:

- To develop a responsive vendor and supplier management web application using React.js.
- To understand and apply React components and state.
- To implement state management using the useState and useEffect hooks.
- To handle API requests using Axios for CRUD operations.
- To implement client-side, page-based navigation using React Router.
- To build a REST API backend using Spring Boot and Spring Data JPA for data persistence.
- To design a purchase workflow covering requests, quotations, purchase orders, and request tracking.
- To provide separate pages and access for the admin and vendor roles.
- To enforce strong password rules when a vendor account is created.

3. Technologies Used

Technology	Purpose
React.js	Frontend UI development
Vite	Build tool and development server (port 5173)
JavaScript (ES6+)	Application logic
HTML5	Page structure
CSS3	Component styling, with a separate stylesheet for each page
Axios	HTTP requests to the backend API
React Router DOM	Page navigation / routing
Spring Boot (Java)	Backend REST API (port 8080)
Spring Data JPA	Database access through repositories
Relational database	Persistent storage of vendors, suppliers, and products
Browser localStorage	Keeps the logged-in username and role on the client

4. System Requirements

Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB minimum
- Storage: 2 GB free space

Software Requirements

- Windows 10/11 (macOS or Linux also supported)
- Node.js and npm
- Java Development Kit (JDK) and a build tool for the Spring Boot project
- A code editor / IDE for the frontend and backend
- A relational database server
- Web Browser (Chrome/Edge/Firefox)
- Backend running at http://localhost:8080 and frontend at http://localhost:5173

5. Project Structure

Frontend

```
frontend/
│
├── public/
│
├── src/
│   ├── Pages/
│   │   ├── Login.jsx
│   │   ├── AdminDashboard.jsx
│   │   ├── Vendors.jsx
│   │   ├── VendorList.jsx
│   │   ├── Suppliers.jsx
│   │   ├── Products.jsx
│   │   ├── Quotations.jsx
│   │   ├── PurchaseOrders.jsx
│   │   ├── Reports.jsx
│   │   ├── UserDashboard.jsx
│   │   └── PurchaseRequests.jsx
│   │      (each page has a matching .css file)
│   │
│   ├── App.jsx
│   ├── main.jsx
│   └── index.css
│
├── index.html
├── package.json
└── vite.config.js
```

Backend

```
backend/src/main/java/com/example/backend/
│
├── controller/   REST controllers (products, vendors, suppliers, auth)
├── service/      Business logic (e.g. ProductService)
├── repository/   Spring Data JPA repositories (e.g. ProductRepository)
└── entity/       Entities (Product, Vendor, Supplier)
```

6. Modules

6.1 Login Module

Provides authentication for two roles. The user selects Admin or Vendor and enters a username and password. Admin credentials are checked in the application, while vendor credentials are verified through the backend login API. On success, the username and role are stored in localStorage and the user is redirected to the Admin Dashboard or the Vendor Dashboard. The page also links to vendor registration.

6.2 Admin Dashboard Module

Acts as the central hub after admin login. It shows live counts of vendors, suppliers, products, and pending requests, followed by a workflow of steps (Vendors, Suppliers, Products, Purchasing, and Reports) with buttons to open each page. A logout option clears the stored login details.

6.3 Vendor Management Module

Allows the admin to register a vendor with name, company, email, phone, address, username, and password. The password must be at least 8 characters with an uppercase letter, a lowercase letter, a number, and a special character, must not contain the username, and must be confirmed. Registered vendors are shown as cards, can be edited in a pop-up form (including the ACTIVE / INACTIVE status), and can be deleted after confirmation.

6.4 Supplier Management Module

Allows the admin to add new suppliers (name, company, email, phone, address), view all suppliers as cards with clickable email and phone, search by supplier or company, edit supplier details, and delete suppliers.

6.5 Product Management Module

Allows the admin to add products with a name, price, and quantity. Products are shown as cards with a gadget icon, price, and stock status, and can be searched, edited, or deleted. Summary tiles show the number of products, total units in stock, and total stock value.

6.6 Vendor Dashboard (Purchase Request) Module

Lets a logged-in vendor browse the available products with price and available quantity, choose a request quantity that cannot exceed the stock, and place a purchase request. While a request is pending, the button changes to Request Pending. A side panel lists the vendor's own requests with colour-coded status.

6.7 Quotation Module

Allows the admin to see pending purchase requests, select one, and view the request details with the total amount (price multiplied by requested quantity). The admin selects a supplier from the supplier list and confirms the quotation, which saves the supplier name, total amount, and quotation status against the product.

6.8 Purchase Orders Module

Lists the pending purchase requests with the requested and available quantities. The admin can confirm a request, which reduces the product stock and marks the request CONFIRMED, or reject it, which marks the request REJECTED.

6.9 My Purchase Requests Module

Allows the vendor to track every request with a progress tracker (Request placed, Admin review, Quotation), a status tag, and filter tabs for All, Pending, Confirmed, and Rejected. For confirmed requests, the quotation details (supplier, price per item, quantity, and total amount) are shown. The page refreshes automatically every 5 seconds.

6.10 Reports Module

Provides reports on the operations of the system and is opened from the Admin Dashboard.

7. Functional Requirements

Login
The user selects a role and enters a username and password. Valid credentials redirect the admin to the Admin Dashboard and the vendor to the Vendor Dashboard; invalid credentials show an error message and keep the user on the Login page.

Register Vendor
The admin enters the vendor's details and a password that satisfies the password rules. The submit button stays disabled until all rules are met and both passwords match.

View / Edit / Delete Vendor
The admin can view all vendors fetched from the API, edit a selected vendor including the ACTIVE / INACTIVE status, or delete a vendor after confirming.

Add / View / Edit / Delete Supplier
The admin can enter a supplier's name, company, email, phone, and address to add a new record, view all suppliers, edit a selected supplier, or delete a supplier.

Add / View / Edit / Delete Product
The admin can enter a product name, price, and quantity to add a product, view all products, edit a selected product, or delete a product.

Place Purchase Request
The vendor selects a product and a quantity not greater than the available stock, and the system saves the request against the product with the status PENDING.

Create Quotation
The admin selects a pending request and a supplier, and the system calculates the total amount and marks the quotation as CONFIRMED.

Confirm / Reject Purchase Request
The admin confirms a pending request, which reduces the available stock, or rejects it. The decision is shown to the vendor.

Track Requests
The vendor views the status of each request and, once confirmed, the quotation details. The page refreshes automatically.

8. React Concepts Used

This project demonstrates the following React concepts:

Components

```
const AdminDashboard = () => {
  return (
    <div className="ad-page">...</div>
  );
};
```

State

```
const [products, setProducts] = useState([]);
const [quantities, setQuantities] = useState({});
```

useEffect

```
useEffect(() => {
  getProducts();
}, []);
```

Event Handling

```
const handleLogin = async (e) => {
  e.preventDefault();
  if (role === "") {
    setError("Please select a role.");
    return;
  }
  ...
};
```

Conditional Rendering

```
{myRequests.length === 0 ? (
  <div>No purchase requests yet.</div>
) : (
  myRequests.map((product) => ...)
)}
```

Navigation with useNavigate

```
const navigate = useNavigate();
navigate("/admin");
```

Automatic Refresh with setInterval

```
useEffect(() => {
  const interval = setInterval(() => {
    getProducts();
  }, 5000);
  return () => clearInterval(interval);
}, [getProducts]);
```

9. Routing

The project uses React Router DOM for client-side navigation between pages.

URL	Component	Purpose
/	Login	Admin and vendor login page
/admin	AdminDashboard	Admin navigation hub
/vendors	Vendors	Vendor registration

URL	Component	Purpose
/vendor-list	VendorList	View, edit, and delete vendors
/suppliers	Suppliers	Manage suppliers (CRUD)
/products	Products	Manage products (CRUD)
/quotations	Quotations	Create quotations for requests
/purchase-orders	PurchaseOrders	Confirm or reject requests
/reports	Reports	View reports
/vendor	UserDashboard	Vendor dashboard and request placing
/purchase-requests	PurchaseRequests	Vendor request tracking

Example:

```
<Route path="/" element={<Login />} />
<Route path="/admin" element={<AdminDashboard />} />
<Route path="/vendors" element={<Vendors />} />
<Route path="/vendor-list" element={<VendorList />} />
<Route path="/suppliers" element={<Suppliers />} />
<Route path="/products" element={<Products />} />
<Route path="/quotations" element={<Quotations />} />
<Route path="/purchase-orders" element={<PurchaseOrders />} />
<Route path="/vendor" element={<UserDashboard />} />
```

10. API Integration

Axios is used to communicate with the Spring Boot REST API running at http://localhost:8080/api. The backend allows requests from the frontend origin http://localhost:5173 through CORS.

Authentication endpoint

- POST /api/auth/login: Verify a vendor's username and password and return the role

Vendors endpoint

- POST /api/vendors/register: Register a new vendor
- GET /api/vendors: Retrieve all vendors
- PUT /api/vendors/{id}: Update a vendor
- DELETE /api/vendors/{id}: Delete a vendor

Suppliers endpoint

- GET /api/suppliers: Retrieve all suppliers
- POST /api/suppliers: Add a new supplier
- PUT /api/suppliers/{id}: Update a supplier
- DELETE /api/suppliers/{id}: Delete a supplier

Products endpoint

- GET /api/products: Retrieve all products
- GET /api/products/{id}: Retrieve one product
- POST /api/products: Add a new product
- PUT /api/products/{id}: Update a product
- DELETE /api/products/{id}: Delete a product
- PUT /api/products/{id}/request: Place a purchase request (vendor)
- PUT /api/products/{id}/quotation-confirm: Confirm a quotation with a supplier name (admin)
- PUT /api/products/{id}/confirm: Confirm a purchase request and reduce stock (admin)
- PUT /api/products/{id}/reject: Reject a purchase request (admin)

11. Database Design

Data is stored in a relational database accessed through Spring Data JPA. The three main resources are described below.

Vendors

Field	Type	Description
id	Long	Unique identifier (auto-generated)
vendorName	String	Name of the vendor
companyName	String	Vendor's company
email	String	Email address
phone	String	Phone number
address	String	Address
username	String	Login username
password	String	Login password
status	String	ACTIVE or INACTIVE

Suppliers

Field	Type	Description
id	Long	Unique identifier (auto-generated)
supplierName	String	Name of the supplier
companyName	String	Supplier's company
email	String	Email address
phone	String	Phone number
address	String	Address

Products

Field	Type	Description
id	Long	Unique identifier (auto-generated)
productName	String	Name of the product
price	Double	Price per item
quantity	Integer	Quantity available in stock
requestedQuantity	Integer	Quantity in the current purchase request
requestedBy	String	Username of the vendor who made the request
requestStatus	String	NONE, PENDING, CONFIRMED, or REJECTED
quotationStatus	String	NONE or CONFIRMED
supplierName	String	Supplier chosen in the quotation
quotationTotal	Double	Total amount (price x requested quantity)

12. Application Flow

User

↓

React UI (Login / Dashboards / Vendors / Suppliers / Products / Quotations / Purchase Orders / Requests)

↓

React Components (state via useState / useEffect)

↓

Axios (HTTP requests)

↓

Spring Boot REST Controllers

↓

Service Layer

↓

Spring Data JPA Repositories

↓

Database

Purchase Request Workflow

1. The vendor logs in, selects a product and quantity, and places a request. The request status becomes PENDING.
2. The admin opens the Quotations page, selects the pending request, chooses a supplier, and confirms the quotation.

3. The admin opens the Purchase Orders page and confirms the request, which reduces the stock and marks it CONFIRMED, or rejects it, which marks it REJECTED.
4. The vendor opens My Purchase Requests to see the result and, for confirmed requests, the quotation details.

13. Testing

Test Case	Input	Expected Result
Admin Login	Valid admin credentials	Redirected to the Admin Dashboard
Login	Invalid credentials	Error shown, stays on the Login page
Login	No role selected	Message asks the user to select a role
Vendor Login	Valid vendor credentials	Redirected to the Vendor Dashboard
Register Vendor	Weak password	Submit stays disabled, rules shown
Register Vendor	Valid details and strong password	Vendor registered and listed
Edit Vendor	Change status to INACTIVE	Vendor card shows INACTIVE
Add Supplier	Valid supplier details	Supplier added and listed
Add Product	Valid name, price, quantity	Product added and listed
Place Request	Quantity greater than stock	Error shown, request not placed
Place Request	Valid quantity	Request saved with status PENDING
Create Quotation	No supplier selected	Message asks to select a supplier
Create Quotation	Pending request and a supplier	Quotation confirmed with total amount
Confirm Request	Pending request	Stock reduced, status CONFIRMED
Reject Request	Pending request	Status REJECTED
Track Requests	Confirmed request	Vendor sees the quotation details
Logout	Click Logout	Stored login details cleared, back to Login

14. Advantages

- Simple and easy-to-use interface for administrators and vendors
- Centralized management of vendors, suppliers, and products
- Complete purchase workflow from request to quotation to order
- Vendors can track their requests without contacting the admin
- Strong password rules during vendor registration
- Reusable React components and a separate stylesheet for each page
- Fast development and hot-reload with Vite
- Clear separation of pages via React Router and of layers in the Spring Boot backend

15. Future Enhancements

- Secure authentication (hashed passwords with BCrypt, JWT-based sessions)
- Move the admin login from the frontend to the backend
- Store purchase requests in a separate table so that several vendors can request the same product
- Search and filter options for vendors, plus pagination for large lists
- Low-stock alerts and automatic reorder suggestions
- Email or SMS notifications when a request is confirmed or rejected
- Downloadable reports and invoices (PDF / Excel)
- Role-based access for multiple staff accounts
- Deployment to a cloud hosting platform

16. Conclusion

The Vendor and Supplier Management System demonstrates how React.js and Spring Boot can be combined to build a functional, real-world administrative application. The project provides practical experience with component-based UI design, state management, client-side routing, REST API design, and database integration for CRUD operations.

The system can be further extended with proper authentication, a more flexible request model, and additional features such as notifications and reporting, making it suitable for real deployment in a retail environment.